

对指针有效，与指针所指的对象无关：

```
const int *p = nullptr;      // p 是一个指向整型常量的指针
constexpr int *q = nullptr; // q 是一个指向整数的常量指针
```

p 和 q 的类型相差甚远，p 是一个指向常量的指针，而 q 是一个常量指针，其中的关键在于 `constexpr` 把它所定义的对象置为了顶层 `const`（参见 2.4.3 节，第 57 页）。

与其他常量指针类似，`constexpr` 指针既可以指向常量也可以指向一个非常量：

```
constexpr int *np = nullptr; // np 是一个指向整数的常量指针，其值为空
int j = 0;
constexpr int i = 42;        // i 的类型是整型常量
// i 和 j 都必须定义在函数体之外
constexpr const int *p = &i; // p 是常量指针，指向整型常量 i
constexpr int *pl = &j;      // pl 是常量指针，指向整数 j
```

2.4.4 节练习

练习 2.32：下面的代码是否合法？如果非法，请设法将其修改正确。

```
int null = 0, *p = null;
```

2.5 处理类型

随着程序越来越复杂，程序中用到的类型也越来越复杂，这种复杂性体现在两个方面。一是一些类型难于“拼写”，它们的名字既难记又容易写错，还无法明确体现其真实目的和含义。二是有时候根本搞不清到底需要的类型是什么，程序员不得不回过头去从程序的上下文中寻求帮助。

2.5.1 类型别名

类型别名（type alias）是一个名字，它是某种类型的同义词。使用类型别名有很多好处，它让复杂的类型名字变得简单明了、易于理解和使用，还有助于程序员清楚地知道使用该类型的真实目的。

有两种方法可用于定义类型别名。传统的方法是使用关键字 `typedef`：

```
typedef double wages;      // wages 是 double 的同义词
typedef wages base, *p;    // base 是 double 的同义词，p 是 double* 的同义词
```

68 其中，关键字 `typedef` 作为声明语句中的基本数据类型（参见 2.3 节，第 45 页）的一部分出现。含有 `typedef` 的声明语句定义的不再是变量而是类型别名。和以前的声明语句一样，这里的声明符也可以包含类型修饰，从而也能由基本数据类型构造出复合类型来。

**C++
11**

新标准规定了一种新的方法，使用别名声明（alias declaration）来定义类型的别名：

```
using SI = Sales_item;    // SI 是 Sales_item 的同义词
```

这种方法用关键字 `using` 作为别名声明的开始，其后紧跟别名和等号，其作用是把等号左侧的名字规定成等号右侧类型的别名。

类型别名和类型的名字等价，只要是类型的名字能出现的地方，就能使用类型别名：

```
wages hourly, weekly;    // 等价于 double hourly, weekly;
```

```
SI item; // 等价于 Sales_item item
```

指针、常量和类型别名



如果某个类型别名指代的是复合类型或常量，那么把它用到声明语句里就会产生意想不到的后果。例如下面的声明语句用到了类型 `pstring`，它实际上是类型 `char*` 的别名：

```
typedef char *pstring;
const pstring cstr = 0; // cstr 是指向 char 的常量指针
const pstring *ps;     // ps 是一个指针，它的对象是指向 char 的常量指针
```

上述两条声明语句的基本数据类型都是 `const pstring`，和过去一样，`const` 是对给定类型的修饰。`pstring` 实际上是指向 `char` 的指针，因此，`const pstring` 就是指向 `char` 的常量指针，而非指向常量字符的指针。

遇到一条使用了类型别名的声明语句时，人们往往会错误地尝试把类型别名替换成它本来的样子，以理解该语句的含义：

```
const char *cstr = 0; // 是对 const pstring cstr 的错误理解
```

再强调一遍：这种理解是错误的。声明语句中用到 `pstring` 时，其基本数据类型是指针。可是用 `char*` 重写了声明语句后，数据类型就变成了 `char`，`*` 成为了声明符的一部分。这样改写的结果是，`const char` 成了基本数据类型。前后两种声明含义截然不同，前者声明了一个指向 `char` 的常量指针，改写后的形式则声明了一个指向 `const char` 的指针。

2.5.2 auto 类型说明符



编程时常常需要把表达式的值赋给变量，这就要求在声明变量的时候清楚地知道表达式的类型。然而要做到这一点并非那么容易，有时甚至根本做不到。为了解决这个问题，C++11 新标准引入了 **auto** 类型说明符，用它就能让编译器替我们去分析表达式所属的类型。和原来那些只对应一种特定类型的说明符（比如 `double`）不同，auto 让编译器通过初始值来推算变量的类型。显然，auto 定义的变量必须有初始值：

C++
11
69

```
// 由 val1 和 val2 相加的结果可以推断出 item 的类型
auto item = val1 + val2; // item 初始化为 val1 和 val2 相加的结果
```

此处编译器将根据 `val1` 和 `val2` 相加的结果来推断 `item` 的类型。如果 `val1` 和 `val2` 是类 `Sales_item`（参见 1.5 节，第 17 页）的对象，则 `item` 的类型就是 `Sales_item`；如果这两个变量的类型是 `double`，则 `item` 的类型就是 `double`，以此类推。

使用 `auto` 也能在一条语句中声明多个变量。因为一条声明语句只能有一个基本数据类型，所以该语句中所有变量的初始基本数据类型都必须一样：

```
auto i = 0, *p = &i; // 正确：i 是整数、p 是整型指针
auto sz = 0, pi = 3.14; // 错误：sz 和 pi 的类型不一致
```

复合类型、常量和 auto

编译器推断出来的 `auto` 类型有时候和初始值的类型并不完全一样，编译器会适当地改变结果类型使其更符合初始化规则。

首先，正如我们所熟知的，使用引用其实是使用引用的对象，特别是当引用被用作初始值时，真正参与初始化的其实是引用对象的值。此时编译器以引用对象的类型作为 `auto` 的类型：

```
int i = 0, &r = i;
auto a = r;           // a是一个整数(r是i的别名,而i是一个整数)
```

其次, auto 一般会忽略掉顶层 const (参见 2.4.3 节, 第 57 页), 同时底层 const 则会保留下来, 比如当初始值是一个指向常量的指针时:

```
const int ci = i, &cr = ci;
auto b = ci;          // b是一个整数(ci的顶层const特性被忽略掉了)
auto c = cr;          // c是一个整数(cr是ci的别名,ci本身是一个顶层const)
auto d = &i;           // d是一个整型指针(整数的地址就是指向整数的指针)
auto e = &ci;          // e是一个指向整数常量的指针(对常量对象取地址是一种底层const)
```

如果希望推断出的 auto 类型是一个顶层 const, 需要明确指出:

```
const auto f = ci;      // ci的推演类型是int, f是const int
```

还可以将引用的类型设为 auto, 此时原来的初始化规则仍然适用:

```
auto &g = ci;           // g是一个整型常量引用, 绑定到ci
auto &h = 42;           // 错误: 不能为非常量引用绑定字面值
const auto &j = 42;      // 正确: 可以为常量引用绑定字面值
```

70 设置一个类型为 auto 的引用时, 初始值中的顶层常量属性仍然保留。和往常一样, 如果我们给初始值绑定一个引用, 则此时的常量就不是顶层常量了。

要在一条语句中定义多个变量, 切记, 符号 & 和 * 只从属于某个声明符, 而非基本数据类型的一部分, 因此初始值必须是同一种类型:

```
auto k = ci, &l = i;     // k是整数, l是整型引用
auto &m = ci, *p = &ci;  // m是对整型常量的引用, p是指向整型常量的指针
// 错误: i的类型是int而&ci的类型是const int
auto &n = i, *p2 = &ci;
```

2.5.2 节练习

练习 2.33: 利用本节定义的变量, 判断下列语句的运行结果。

```
a = 42; b = 42; c = 42;
d = 42; e = 42; g = 42;
```

练习 2.34: 基于上一个练习中的变量和语句编写一段程序, 输出赋值前后变量的内容, 你刚才的推断正确吗? 如果不对, 请反复研读本节的示例直到你明白错在何处为止。

练习 2.35: 判断下列定义推断出的类型是什么, 然后编写程序进行验证。

```
const int i = 42;
auto j = i; const auto &k = i; auto *p = &i;
const auto j2 = i, &k2 = i;
```



2.5.3 decltype 类型指示符



有时会遇到这种情况: 希望从表达式的类型推断出要定义的变量的类型, 但是不想用该表达式的值初始化变量。为了满足这一要求, C++11 新标准引入了第二种类型说明符 **decltype**, 它的作用是选择并返回操作数的数据类型。在此过程中, 编译器分析表达式并得到它的类型, 却不实际计算表达式的值:

```
decltype(f()) sum = x;    // sum的类型就是函数f的返回类型
```


编译器并不实际调用函数 `f`，而是使用当调用发生时 `f` 的返回值类型作为 `sum` 的类型。换句话说，编译器为 `sum` 指定的类型是什么呢？就是假如 `f` 被调用的话将会返回的那个类型。

`decltype` 处理顶层 `const` 和引用的方式与 `auto` 有些许不同。如果 `decltype` 使用的表达式是一个变量，则 `decltype` 返回该变量的类型（包括顶层 `const` 和引用在内）：

71

```
const int ci = 0, &cj = ci;
decltype(ci) x = 0;      // x 的类型是 const int
decltype(cj) y = x;      // y 的类型是 const int&, y 绑定到变量 x
decltype(cj) z;          // 错误: z 是一个引用, 必须初始化
```

因为 `cj` 是一个引用，`decltype(cj)` 的结果就是引用类型，因此作为引用的 `z` 必须被初始化。

需要指出的是，引用从来都作为其所指对象的同义词出现，只有用在 `decltype` 处是一个例外。

decltype 和引用

如果 `decltype` 使用的表达式不是一个变量，则 `decltype` 返回表达式结果对应的类型。如 4.1.1 节（第 120 页）将要介绍的，有些表达式将向 `decltype` 返回一个引用类型。一般来说当这种情况发生时，意味着该表达式的结果对象能作为一条赋值语句的左值：

```
// decltype 的结果可以是引用类型
int i = 42, *p = &i, &r = i;
decltype(r + 0) b;    // 正确: 加法的结果是 int, 因此 b 是一个 (未初始化的) int
decltype(*p) c;       // 错误: c 是 int&, 必须初始化
```

因为 `r` 是一个引用，因此 `decltype(r)` 的结果是引用类型。如果想让结果类型是 `r` 所指的类型，可以把 `r` 作为表达式的一部分，如 `r+0`，显然这个表达式的结果将是一个具体值而非一个引用。

另一方面，如果表达式的内容是解引用操作，则 `decltype` 将得到引用类型。正如我们所熟悉的那样，解引用指针可以得到指针所指的对象，而且还能给这个对象赋值。因此，`decltype(*p)` 的结果类型就是 `int&`，而非 `int`。

`decltype` 和 `auto` 的另一处重要区别是，`decltype` 的结果类型与表达式形式密切相关。有一种情况需要特别注意：对于 `decltype` 所用的表达式来说，如果变量名加上了一对括号，则得到的类型与不加括号时会有不同。如果 `decltype` 使用的是一个不加括号的变量，则得到的结果就是该变量的类型；如果给变量加上了一层或多层括号，编译器就会把它当成是一个表达式。变量是一种可以作为赋值语句左值的特殊表达式，所以这样的 `decltype` 就会得到引用类型：

```
// decltype 的表达式如果是加上了括号的变量，结果将是引用
decltype((i)) d;      // 错误: d 是 int&, 必须初始化
decltype(i) e;        // 正确: e 是一个 (未初始化的) int
```



切记：`decltype((variable))`（注意是双层括号）的结果永远是引用，而 `decltype(variable)` 结果只有当 `variable` 本身就是一个引用时才是引用。

72

2.5.3 节练习

练习 2.36: 关于下面的代码, 请指出每一个变量的类型以及程序结束时它们各自的值。

```
int a = 3, b = 4;
decltype(a) c = a;
decltype((b)) d = a;
++c;
++d;
```

练习 2.37: 赋值是会产生引用的一类典型表达式, 引用的类型就是左值的类型。也就是说, 如果 `i` 是 `int`, 则表达式 `i=x` 的类型是 `int&`。根据这一特点, 请指出下面的代码中每一个变量的类型和值。

```
int a = 3, b = 4;
decltype(a) c = a;
decltype(a = b) d = a;
```

练习 2.38: 说明由 `decltype` 指定类型和由 `auto` 指定类型有何区别。请举出一个例子, `decltype` 指定的类型与 `auto` 指定的类型一样; 再举一个例子, `decltype` 指定的类型与 `auto` 指定的类型不一样。



2.6 自定义数据结构

从最基本的层面理解, 数据结构是把一组相关的数据元素组织起来然后使用它们的策略和方法。举一个例子, 我们的 `Sales_item` 类把书本的 ISBN 编号、售出量及销售收入等数据组织在了一起, 并且提供诸如 `isbn` 函数、`>>`、`<<`、`+`、`+=` 等运算在内的一系列操作, `Sales_item` 类就是一个数据结构。

C++ 语言允许用户以类的形式自定义数据类型, 而库类型 `string`、`istream`、`ostream` 等也都是以类的形式定义的, 就像第 1 章的 `Sales_item` 类型一样。C++ 语言对类的支持甚多, 事实上本书的 III 部分和 IV 部分都将大篇幅地介绍与类有关的知识。尽管 `Sales_item` 类非常简单, 但是要想给出它的完整定义可在第 14 章介绍自定义运算符之后。



2.6.1 定义 `Sales_data` 类型

尽管我们还写不出完整的 `Sales_item` 类, 但是可以尝试着把那些数据元素组织到一起形成一个简单点儿的类。初步的想法是用户能直接访问其中的数据元素, 也能实现一些基本的操作。

既然我们筹划的这个数据结构不带有任何运算功能, 不妨把它命名为 `Sales_data` 以示与 `Sales_item` 的区别。 `Sales_data` 初步定义如下:

73

```
struct Sales_data {
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

我们的类以关键字 **struct** 开始, 紧跟着类名和类体 (其中类体部分可以为空)。类体由